# NYCU-EE ICLAB – Spring 2026

## Lab12 Exercise:

## Design: Diffusion Model APR

### Data Preparation

- Extract the test data from TA's directory:

  **% tar xvf ~iclabTA01/Lab12_26S.tar**

- The extracted LAB directory contains:
  - Exercise/

### Design Description

**Diffusion Model (DM)** is a generative model architecture designed to learn how to gradually remove noise from corrupted data and reconstruct the original sample. Unlike conventional discriminative models that directly map an input to a label or prediction, diffusion models learn the reverse process of a predefined noising procedure. By progressively denoising a noisy image step by step, the model can generate high-quality outputs and capture complex data distributions. For example, as illustrated in Fig. 1.

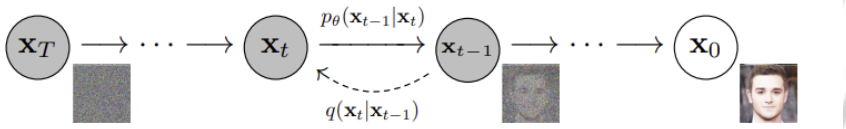



Fig. 1. The Diffusion Model example

In a traditional diffusion process, noise is gradually added to an input sample $x_0$ over multiple time steps until the data becomes nearly pure noise. The model is then trained to learn the reverse process, which **predicts and removes the added noise** at each step. If the noisy sample at time step $t$ is denoted as $x_t$, the reverse process can be written as learning a function that estimates the noise component $\epsilon_\theta(x_t, t)$, so that the original sample can be progressively recovered. This procedure allows the model to generate or reconstruct images through iterative refinement, making diffusion models effective for image synthesis and restoration tasks.

In Lab05, you have already designed and implemented a **simplified W4A8 Diffusion Model** architecture. In this lab, you will finish the **backend flow (APR)** for **Lab05 netlist files** and use the **IR drop** and **Power Analysis** tool for your layout. You don't have to write RTL design again.

## I/O Description

| Input | Bit Width | Definition and Description |
|---|---|---|
| clk | 1 | Clock |
| rst_n | 1 | Asynchronous active-low reset |
| i_valid | 1 | High when sending weight/image/iteration/mode values. |
| i_iter | 3 | The number of denoising iterations for the input image is transmitted through i_iter and is valid only in the first cycle of the input sequence. The iteration value ranges from 1 to 7. One iteration refers to performing the complete denoising pipeline once. |
| i_mode | 2 | The mode of interpolation for the input image is transmitted through i_mode and is valid only in the first cycle of the input sequence. |
| i_data | 8 | When "i_valid" is high, i_data is sending the image data in raster scan order, where it contains 8 bits unsigned integer value. |
| i_weight | 4 | When "i_valid" is high, i_weight sends the value of weight. |
| **Output** | **Bit Width** | **Definition and Description** |
| o_valid | 1 | High only when o_data is valid. It cannot be overlapped with i_valid . |
| o_data | 8 | It will output the final result of the diffusion model in in raster scan order after finishing the calculation. Each o_data contains 8 bits unsigned value for one denoised pixel value. Output doesn't need to be continuous. |

## Layout Specification

1. CHIP.sdc clock period can be defined by yourself.

2. Floorplanning

   - Core size: Defined by you
   - Core to IO boundary: Each side must be larger than 100

3. Power Planning

   - Core Ring
     - Top & Bottom: metal layer must be odd and width is fixed to 9
     - Left & Right: metal layer must be even and width is fixed to 9
     - Each side must be wire group, interleaving, and at least 5 pairs
   - Stripes
     - Horizontal: metal layer must be odd and width at least 2
     - Vertical: metal layer must be even and width at least 2

- The maximum distance between two stripes or stripe and edge should be less than 200

4. Timing Analysis

   - Timing Slack: No negative slacks after setup/hold time analysis (Post-Route stage)
   - Design Rule Violation (DRV): The DRV of (fanout, cap, tran) should be all 0 after setup/hold time analysis

5. Design Verification Result

   - LVS: No LVS violations after "verify connectivity"
   - DRC: No DRC violations after "verify DRC"
   - Note: Remember to check DRC / LVS again after placing the fillers.
     Although in normal cases, if DRC and LVS are verified after nanoRoute, no more DRC / LVS will be produced during postRoute optimization and adding core filler. However, some special cases produce the further DRC and LVS.
     For example: inserting a filler that is isolated from all other cells and power lines, then open LVS occurs. Thus, be sure to verify all the specs above after performing all APR steps.

6. Post Layout Simulation: The post-simulation cannot include any timing violations without the +notimingcheck command.

7. Rail Analysis:

   - VCC Threshold set to 1.7
   - GND Threshold set to 0.1
   - No IR drop is allowed larger than 1mV

## Grading Policy

- You should meet all the Layout Specification and File Specification above, pass post simulation without any timing violation, and within tolerable IR drop to pass the demo.
- 1de demo grade: 100 points
- 2de demo grade: 70 points.

## Note

1. Complete CHIP.io and CHIP_SHELL.v
2. Copy your Lab05 netlist file and sdc file to 05_APR
3. Get your Memory file (.v/ .db /.lef/ .lib)
4. Do all the flow as in APRI (Lab11) with Layout Specification above
5. Run Power Analysis (Setup & Run)

NCTU OASIS Lab founded in 2003

6. Run Rail Analysis
   - Set PG Library Mode
   - Generate PG Library
   - Setup & Run Rail Analysis
7. Observe the IR Drop to analyze whether IR drop is within 1mV.
   - ➢ Please submit your files under 09_SUBMIT and **a report to the E3 system** before 2026/06/01 (Mon.) 12:00:00:

   - **The report should include screenshots and brief explanations. All necessary information is included in "Lab12_report_example.docx".**
     **Your report need to be named as "Lab12_report_iclabXXX.pdf".**
   - If uploaded files violate the naming rule, you will get 5 deduct point.

   ```
   [Exercise/09_SUBMIT]% ./01_submit
   ```

   After that, you should check the following files under 09_SUBMIT/Lab12_iclabXXX/
   i. filelist_iclabXXX.f
   ii. CHIP_iclabXXX.inn
   iii. CHIP_iclabXXX.inn.dat (this one is a directory)
   iv. CHIP_cts_iclabXXX.sdc
   v. CHIP_iclabXXX.sdc
   vi. CHIP_iclabXXX.sdf
   vii. CHIP_iclabXXX.v
   viii. CHIP_iclabXXX.io
   ix. POST_clk_iclabXXX.txt
   x. Memory file: .v/ .db /.lef/ .lib in 04_MEM

   If you miss any files on the list, you will fail this lab.
   Then use the command like the figure below to check the files are uploaded or not

   ```
   [Exercise/09_SUBMIT]% ./02_check 1st_demo
   ```
8. Self-Verify APR Result!!! Remember to restore and check the checklist.